25/03/26

Ordenação

→ Para volume pequeno de dados o algoritmo / máquina não importa

→ Melhor algoritmo quem entrega mais rápido o resultado correto

→ ordenação interna (RAM only) / interna (usa mem. secundária também)

→ Estável x instável → como é lida com a entrada

→ simples - eficiente → alocação de memória não construída a um tamanho

→ Externa usa mem. secundária como um auxiliar

→ Estável → ordem original é preservada (bom para manter ordem cronológica)

→ Instável → não mantém a ordem

Métodos de Ordenação

ordenado pelo campo 2

- Exemplo ➡ Estável:

Critério

Chave

	Campo 1	Campo 2
Registro 1	4	AAA
Registro 2	3	BBB
Registro 3	5	CCC
Registro 4	5	DDD
Registro 5	4	KKK

Arquivo Original

Ordem de Entrada

Campo 1	Campo 2
3	BBB
4	AAA
4	KKK
5	CCC
5	DDD

Arquivo Classificado

ordenado pelo campo 1

dois 4, então diferem na ordem original

- Exemplo: Bubble sort, Insertion sort , Merge sort, Radix sort e Counting sort

→ No começo a ordenação era feita com arquivos

Métodos de Ordenação

- Exemplo ➡ Instável:

Critério

Chave

	Campo 1	Campo 2
Registro 1	4	AAA
Registro 2	3	BBB
Registro 3	5	CCC
Registro 4	5	DDD
Registro 5	4	KKK

Arquivo Original

Ordem de Entrada

Campo 1	Campo 2
3	BBB
4	KKK
4	AAA
5	DDD
5	CCC

Arquivo Classificado

de qualquer ordem

- Exemplo: Selection Sort, Quick Sort, Heap Sort e Shell Sort

→ Simples → ordem de $O(n^2)$ comparações. Fácil de entender. Para pouca entradas. Menos eficiente

→ Eficiente → ordem de $O(n \log n)$ comp. Operação complexa. Para grandes números de argumentos/entrada também podem ser melhorados

→ métodos mais eficazes → In Situ. Permutação da entrada. Pouca auxiliares, no máximo uma pilha. Algoritmos com muitas chamadas/estruturas auxiliares pode não ser a melhor ideia. (não tipicamente bom o bom)

→ Recursividade tira legibilidade

→ Método para situação especial → mais antes vamos ver lista encadeada

→ Para ordenação as medidas são: Tempo, comparação e troca

- Funções de Complexidades Adotadas
  - $T(n)$: tempo de execução
  - $C(n)$: número de comparações
  - $M(n)$: número de trocas

Selection sort

→ Interno
→ divide vetor em 2 segmentos (ordenado e não ordenado)
→ Inicialmente ordenado é vazio e não ordenado é n
→ vai pegando valores menores do não vetor ordenado (menor valor do não ordenado)

Qual é o valor de $T(n)$?

SelectionSort (A, n)	Tempo: $T(n)$
1 para $i \leftarrow 1$ até $n-1$ faça	$O(n)$
2 $min \leftarrow i$	
3 para $j \leftarrow i+1$ até n faça	$\approx O(n)$
4 se $A[j] < A[min]$ então $min \leftarrow j$	
5 $A[i] \leftarrow A[min]$	

Resultado ⇨ $O(n^2)$

- Selection Sort: Algoritmo

SelectionSort (A, n)	Comparações: $C(n)$
1 para $i \leftarrow 1$ até $n-1$ faça	$O(n)$
2 $min \leftarrow i$	
3 para $j \leftarrow i+1$ até n faça	
4 se $A[j] < A[min]$ então $min \leftarrow j$	$O(n)$
5 $A[i] \leftarrow A[min]$	

Resultado ⇨ $O(n^2)$

SelectionSort (A, n) — **Trocas: $M(n)$**

```
para i ← 1 até n − 1 faça
    min ← i
    para j ← i + 1 até n faça
        se A[j] < A[min] então min ← j
    A[i] ← A[min]                                   O(n)
```

Resultado ➡ **$O(n)$**

O problema está no laço aninhado

→ Instável
→ $O(n^2)$ mesmo para entrada ordenada

Insertion sort

→ divide vetor em 2 segmentos
→ parte ordenada contém o primeiro elemento
→ vai inserindo o 1º elemento do segundo segmento de forma ordenada

/* inicialmente o vetor A é considerado um arquivo classificado com um elemento. */

InsertionSort (A, n)	Tempo: $T(n)$
1 **para** $j \leftarrow 2$ **até** n **faça**	$O(n)$
2 $\quad chave \leftarrow A[j]$	
3 $\quad i \leftarrow j - 1$	
4 $\quad$ **enquanto** $i \geq 1$ **e** $A[i] > chave$ **faça**	$O(n)$
5 $\quad\quad A[i+1] \leftarrow A[i]$	
6 $\quad\quad i \leftarrow i - 1$	
7 $\quad A[i+1] \leftarrow chave$	

Resultado ➡ $O(n^2)$

InsertionSort (A, n)	Comparações: $C(n)$	
1 **para** $j \leftarrow 2$ **até** n **faça**	$O(n)$	$O(n^2)$
2 $\quad chave \leftarrow A[j]$		
3 $\quad i \leftarrow j - 1$		
4 $\quad$ **enquanto** $i \geq 1$ **e** $A[i] > chave$ **faça**	$O(n)$	
5 $\quad\quad A[i+1] \leftarrow A[i]$		
6 $\quad\quad i \leftarrow i - 1$		
7 $\quad A[i+1] \leftarrow chave$		

Resultado ➡ $O(n^2)$

InsertionSort (A, n)	Trocas: $M(n)$	
1 **para** $j \leftarrow 2$ **até** n **faça**		
2 $\quad chave \leftarrow A[j]$	$O(n)$	$O(n^2)$
3 $\quad i \leftarrow j - 1$		
4 $\quad$ **enquanto** $i \geq 1$ **e** $A[i] > chave$ **faça**		
5 $\quad\quad A[i+1] \leftarrow A[i]$	$O(n)$	
6 $\quad\quad i \leftarrow i - 1$		
7 $\quad A[i+1] \leftarrow chave$	$O(n)$	

Resultado ➡ $O(n^2)$

Pois que o
selection sort

→ Eh i ressaltar que estado do conjunto. Então se o estado este mais ordenado ele não trabalha no pior caso de C e M

Insertion Sort: Conclusões

- **Observar o Laço mais interno:**
  - i-ésima iteração: C_i e M_i
    - **Melhor caso:**
    - $C_i(n) = 1$ e $M_i(n) = 1$

    - **Pior caso:**
    - $C_i(n) = i\text{-}1$ e $M_i(n) = i\text{-}1$

Pior caso é zero no melhor caso

- **Observar o Laço externo (n-1)**
  - **Melhor caso:**
    - $C_i(n) = 1 \times n\text{-}1 \Rightarrow C(n)$
    - $M_i(n) = 1 \times n\text{-}1 \Rightarrow M(n)$
  - **Pior caso:**
    - $C_i(n) = i\text{-}1 \times n\text{-}1 \Rightarrow O(n^2)$
    - $M_i(n) = i\text{-}1 \times n\text{-}1 \Rightarrow O(n^2)$

InsertionSort (A, n)

```
para j ← 2 até n faça
    chave ← A[j]
    i ← j − 1
    enquanto i ≥ 1 e A[i] > chave faça
        A[i + 1] ← A[i]
        i ← i − 1
    A[i + 1] ← chave
```

→ Estável
→ C e M dependem da lista dada

→ Possível otimização → uma busca binária para saber onde inserir então $C(n) \to O(n \lg n)$

→ Problema é que vai ter muitas mover elementos entre extremos

Shell Sort

→ Quer resolver o problema do Insertion sort
→ segmentação
→ organizada em h partes
→ cada h é uma região ordenada

■ **Shell Sort**: Exemplo de operação

Índices:	0	1	2	3	4
Inicial	30	14	40	25	3
h=4	3	14	40	25	30
			Incremento (h)		
h=1	3	14	30	25	40
	P1	P2	P3	P4	
	3	14	25	30	40

n importa o algoritmo

Knuth fez melhoramento dos incrementos de 4 para aumentar a eficiência

não sabemos por que é eficiente na ...

→ instável
→ para estados de tamanho médio não eficiente para pequenos

Quick sort

→ Divide problema em outros menores
→ Resultados combinados para obter a solução
→ parte inferior / parte superior

Quick Sort: Operação

- *pivô* (*x*): elemento utilizado para comparação
  - Cálculo Pivo: $v[(esq + dir)/2]$

1. Percorra o vetor a partir da esquerda até que $v[esq] \geq pivô$;
2. Percorra o vetor a partir da direita até que $v[dir] \leq pivô$;
3. Troque $v[dir]$ com $v[esq]$;
4. Processo repetido até critério de parada: apontadores *esq* e *dir* ***ultrapassam*** o pivô;
5. Dividir vetor e repetir processo

← pivô é o ponto chave

Quick Sort: Algoritmo

Avaliação de Comparações

```
void qs(char *item, int esq, int dir)
{ int e, d; char elem, y;
  e = esq; d = dir;
  elem = item[(esq+dir)/2];
  do {
        while(item[e]<elem)
            e++;

        while(elem<item[d])
            d--;

        if(e<=d) {
            y = item[e];
            item[e] = item[d];
            item[d] = y;
            e++;  d--; }
  } while(e<d);
...
}
```

Comparações: $C(n)$

$\approx O(n)$

$\approx O(n)$

$\approx O(n)$

n vezes= $O(n)$

Resultado ➡ $O(n)$


```
void qs(char *item, int esq, int dir)
{ int e, d; char elem, y;
  e = esq; d = dir;
  elem = item[(esq+dir)/2];
  do {
        while(item[e]<elem)
            e++;

        while(elem<item[d])
            d--;

        if(e<=d) {
            y = item[e];
            item[e] = item[d];
            item[d] = y;
            e++;  d--; }
  } while(e<d);
  if(esq<d) qs(item, esq, d);
  if(e<dir) qs(item, e, dir);
}
```

Comparações: $C(n)$

$\approx \log n$ vezes= $O(\log n)$

$\approx \log n$ vezes= $O(\log n)$

Resultado ➡ $O(n \log n)$


```
void qs(char *item, int esq, int dir)
{ int e, d; char elem, y;
  e = esq; d = dir;
  elem = item[(esq+dir)/2];
  do {
        while(item[e]<elem)
            e++;

        while(elem<item[d])
            d--;

        if(e<=d) {
            y = item[e];
            item[e] = item[d];
            item[d] = y;
            e++;  d--; }
  } while(e<d);
  if(esq<d) qs(item, esq, d);
  if(e<dir) qs(item, e, dir);
}
```

Comparações: $C(n)$

$g(n)= O(n)$

$f(n)= O(\log n)$

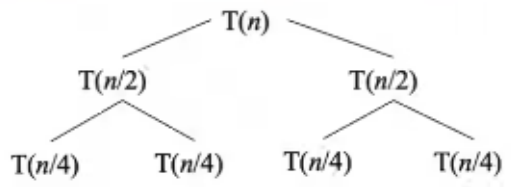
Resultado ➡ $O(f(n)g(n))$ ➡ $O(n \log n)$

→ Dependente do balanceamento do particionamento.

→ Pior caso → $O(n^2)$ longe mais particionamento mal feito

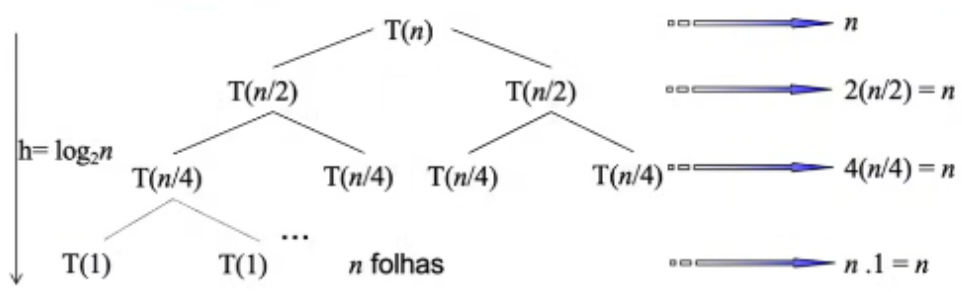
→ Baseado em Memória (memória de tempo)

3. **Segunda Expansão: Divisão e substituição**



- Recorrência: $T(n) = 2T(n / 2) + n$, $T(1) = 1$.

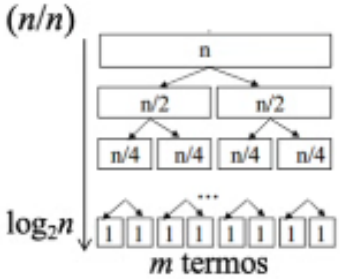
4. ***n* divisões**



- **Generalizando, tem-se:**

  - Assim, o tamanho do arquivo n pode ser visto como uma potência de 2 ($n=2^m$: $m=\log_2 n$)

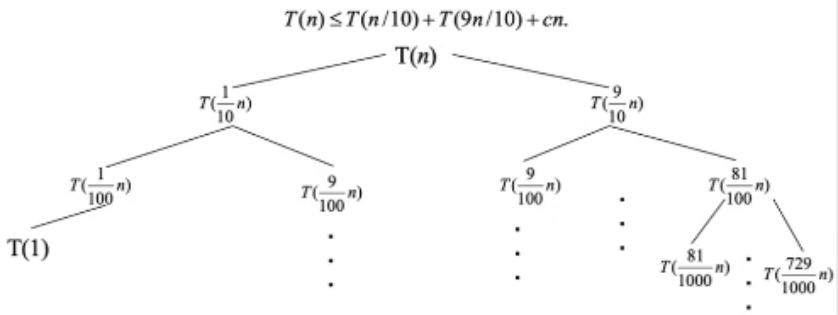
    - $n + 2 * (n/2) + 4 * (n/4) + 16 * (n/16) + \ldots + n * (n/n)$
    - $n + n + n + \ldots + n$(m termos)
    - $T(n*m)$, sendo $m=\log_2 n$.
    - $T(n\log n)$.



- **Caso Médio:** Duas **partes são tratadas como quase iguais**

  - Um exemplo de tamanho de partição: 9 para 1
  - Tem-se: $T(n) = n + T(\left\lfloor \frac{1}{10} n \right\rfloor) + T(\left\lceil \frac{9}{10} n \right\rceil)$.

- **Caso Médio. Solução: Árvore de Recorrência**



Árvore será definida por divisões boas e ruins: *T(n logn).*

→ Método equilibrado e baseado na divisão constante